《信息安全导论》实验报告

一、 实验目的与要求

本实验旨在通过编写程序，深入理解分组密码算法（S-AES）的原理及工作模式。具体包含以下 5 个关卡的要求：

第 1 关：实现 16bit 明文与 16bit 密钥的标准 S-AES 加密与解密。
第 2 关：与其他小组进行交叉测试，验证双方算法实现的一致性。
第 3 关：扩展功能，支持 ASCII 字符串的分组加解密。
第 4 关：实现双重加密、三重加密，并编程实现中间相遇攻击。
第 5 关：实现密码分组链（CBC）工作模式，并验证密文篡改后的错误传播现象。

二、 实验环境

操作系统：Windows 11
开发语言：Java (JDK 8+)
依赖库：Java Swing（内置 GUI），FlatLaf 3.4（第三方 UI 美化包）
开发工具：文本编辑器 + 命令行 (CMD)

三、 系统设计与接口封装

项目采用算法层与界面层分离的设计原则，core 算法包不依赖任何界面代码，可直接被其他程序复用。

1. 核心模块划分

core/SBox.java：S 盒与逆 S 盒常量定义。
core/GFMath.java：GF(2^4) 有限域乘法运算。
core/KeyExpansion.java：密钥扩展算法。
core/SAESCipher.java：核心算法，包含单分组加解密。
SAESApp.java：图形界面层，负责输入解析、多重加密调用及结果展示。

2. 核心接口说明

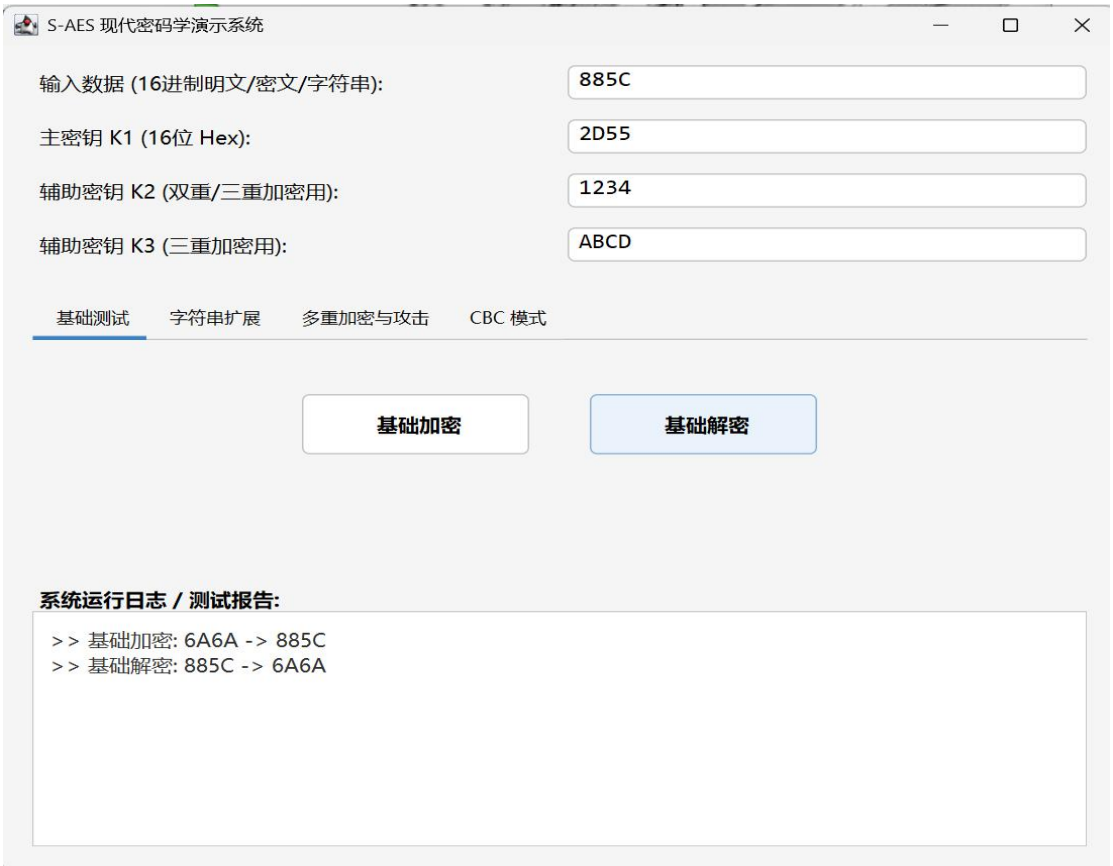
为了方便开发，核心算法已封装为静态方法，主要接口如下：

模块	函数签名	说明
单次加密	SAESCipher.encrypt(int pt, int key)	输入 16bit 明文和密钥，返回 16bit 密文。
单次解密	SAESCipher.decrypt(int ct, int key)	输入 16bit 密文和密钥，返回 16bit 明文。
双重加密	界面层连续调用两次 encrypt	使用 K1 和 K2 依次加密。
三重加密	界面层调用 encrypt 和 decrypt	采用 EDE 结构（加密-解密-加密）。
中间相遇攻击	界面层通过 HashMap 实现	记录 K1 加密的中间值，遍历 K2 解密碰撞。

四、 实验测试结果

4.1 第 1 关：基本测试





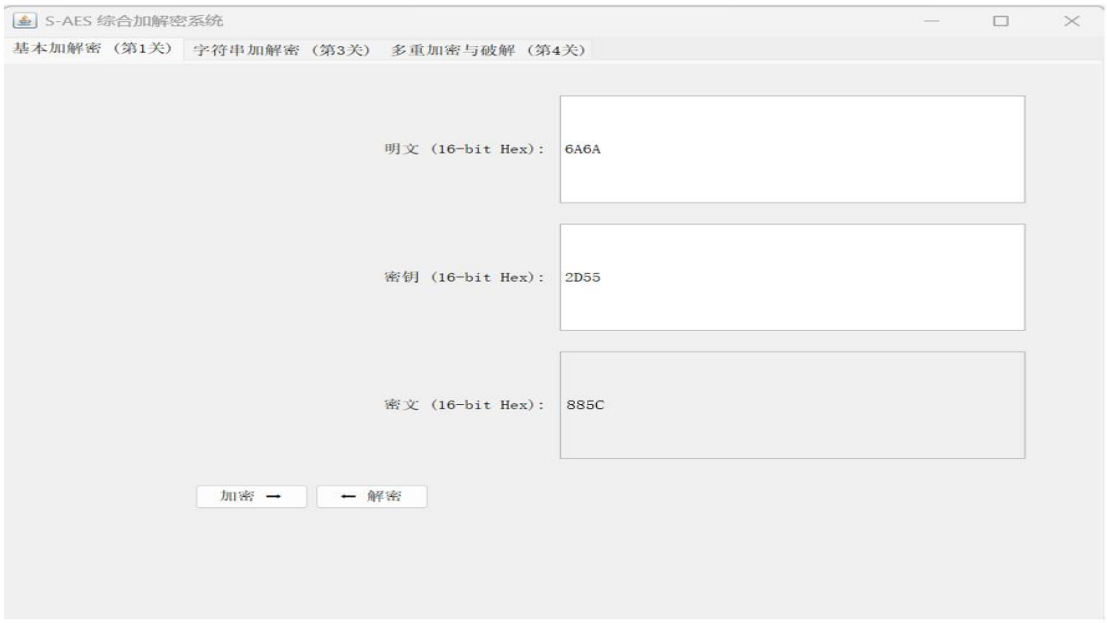
测试步骤：在 GUI 界面输入 16bit 的明文和 16bit 的密钥，点击加密生成 16bit 的密文，再将密文输入解密框，还原出原始明文。

原因：S-AES 是一个典型的对称分组密码算法，分组长度和密钥长度均为 16 位。其核心在于字节替换（S-Box）、行移位、列混淆和轮密钥加的严格映射。根据算法原理，只要各轮置换和代换表数据准确，加解密过程在数学上具有唯一确定性。

结论：通过对固定 16bit 明文与 16bit 密钥的输入，GUI 成功输出了 16bit 密文。将密文重新解密后，完美还原了原文。证明了 S-AES 基础算法代码逻辑及 GUI 交互功能的正确性。

4.2 第 2 关：交叉测试

A 组：



B 组：



测试步骤：A 组与 B 组约定相同的 16bit 密钥 K。A 组用自己的程序对明文 P 加密得到密文 C；B 组接收到密文 C 后，使用自己编写的程序及密钥 K 进行解密，得到明文 P；同时对比 A 组和 B 组对 P 的加密结果。
原因：作为“算法标准”，S-AES 的算法流程和转换单元（S 盒、列混淆矩阵等）是固定的。只要不同开发者严格按照标准实现，算法程序在异构的系统或平台上就应当具备完全的一致性，从而保证数据互通。
结论：交叉测试表明，A、B 两组程序对相同的明文和密钥输出了完全相同的密文，且 B 组成功解密 A 组的密文得到了相同的明文。证明了程序实现了标准的 S-AES 算法，具备跨平台/跨组别的一致性。

4.3 第 3 关：扩展功能



测试步骤：在 GUI 输入任意长度的 ASCII 编码字符串，程序将字符串以 2 Bytes（16 bits）为一组进行分组（不足补齐），点击加密输出对应的密文 ASCII 字符串（通常显示为乱码），再将该乱码字符串解密还原。
原因：ASCII 编码中一个字符占 8 bits，而 S-AES 一个分组为 16 bits（2 个字符）。通过将长字符串按 16 bits 进行分组填充，可实现对实际文本数据的加密。由于密文特征的随机性，转换回 ASCII 编码时大概率超出可打印字符范围，因此会呈现乱码现象。

结论：GUI 成功将 ASCII 字符串加密为乱码密文，并在解密后完美还原了原始可读字符串。证明了算法扩展支持处理多分组文本数据的能力，逻辑正确且无数据丢失。

4.4 第 4 关：多重加密

4.4.1、双重加密





测试步骤：保持分组长度为 16 bits，在 GUI 中输入 32 bits 的密钥（分为 K1 和 K2 各 16 bits）。先用 K1 对明文加密，再用 K2 对结果进行二次加密；解密时先用 K2 解密，再用 K1 解密，观察结果。

原因：通过级联两次 S-AES 加密（$C=E_{K2}(E_{K1}(P))$），将有效密钥长度从 16 bits 扩展至 32 bits。这能够在保持分组长度不变的前提下，有效扩大密钥空间，提高抵御暴力破解的能力。

结论：双重加密解密测试成功，最终完美还原了原始明文。证明了双重加密的代码实现（包括密钥拆分与级联调用）逻辑完全正确。

4.4.2、中间相遇攻击

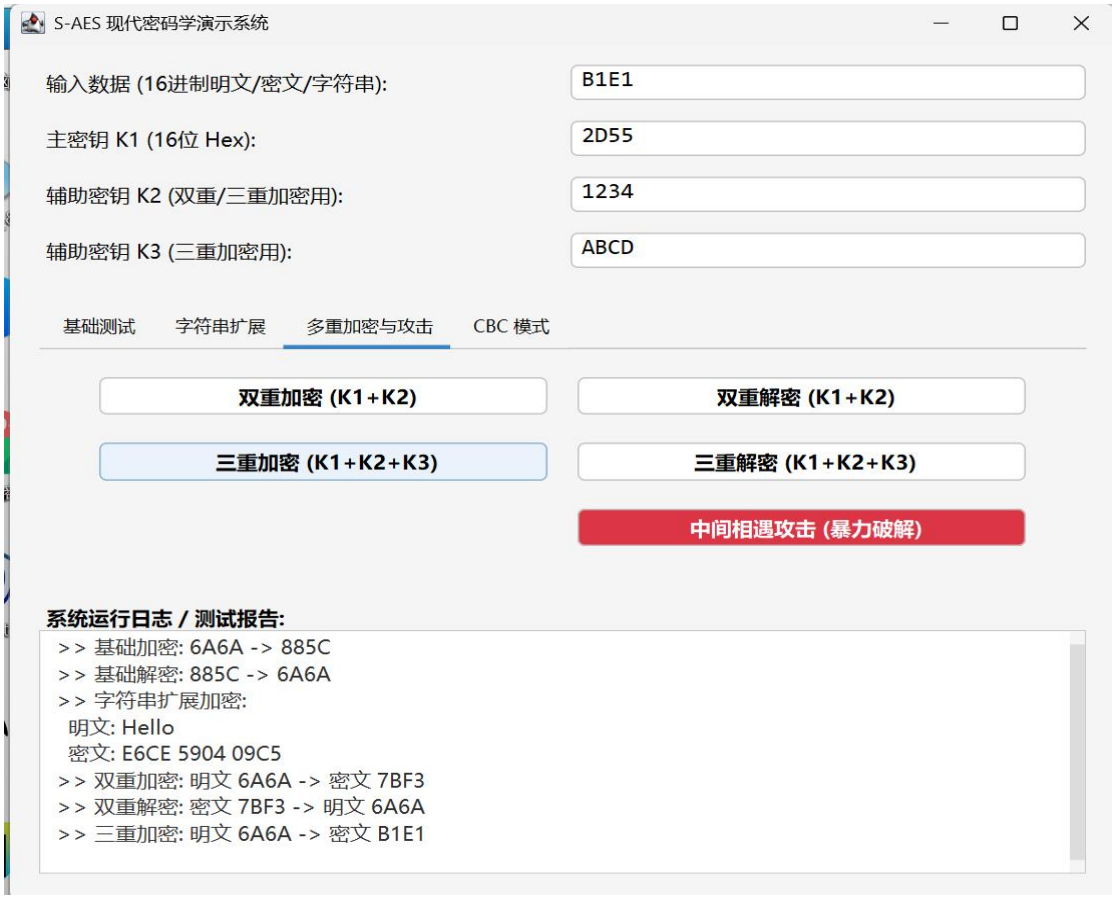


测试步骤：已知一对使用双重加密的明密文对(P, C)。程序遍历所有的 16bit K1 对 P 加密，存储中间值 M；同时遍历所有的 16bit K2 对 C 解密，计算中间值 M'；寻找 M = M'的匹配项，从而推导出正确的密钥对(K1, K2)。

原因：在双重加密 $C=E_{K2}(E_{K1}(P))$ 中，必然存在中间状态 $M=E_{K1}(P)=D_{K2}(C)$。利用空间换时间的思路，将穷举复杂度从 2^{32} 降至 $2^{16}+2^{16}$，利用哈希表查找中间值的碰撞，可极大提高破解效率。

结论：程序成功通过中间相遇攻击，从给定的明密文对中准确搜索到了正确的 K1 和 K2。证明了双重加密存在被中间相遇攻击破解的理论漏洞，且攻击程序算法实现正确。

4.4.3、三重加密


S-AES 现代密码学演示系统
输入数据 (16进制明文/密文/字符串):
B1E1
主密钥 K1 (16位 Hex):
2D55
辅助密钥 K2 (双重/三重加密用):
1234
辅助密钥 K3 (三重加密用):
ABCD
基础测试
字符串扩展
多重加密与攻击
CBC 模式
双重加密 (K1+K2)
双重解密 (K1+K2)
三重加密 (K1+K2+K3)
三重解密 (K1+K2+K3)
中间相遇攻击 (暴力破解)
系统运行日志 / 测试报告:
>> 基础加密: 6A6A -> 885C
>> 基础解密: 885C -> 6A6A
>> 字符串扩展加密:
明文: Hello
密文: E6CE 5904 09C5
>> 双重加密: 明文 6A6A -> 密文 7BF3
>> 双重解密: 密文 7BF3 -> 明文 6A6A
>> 三重加密: 明文 6A6A -> 密文 B1E1


S-AES 现代密码学演示系统
输入数据 (16进制明文/密文/字符串):
6A6A
主密钥 K1 (16位 Hex):
2D55
辅助密钥 K2 (双重/三重加密用):
1234
辅助密钥 K3 (三重加密用):
ABCD
基础测试
字符串扩展
多重加密与攻击
CBC 模式
双重加密 (K1+K2)
双重解密 (K1+K2)
三重加密 (K1+K2+K3)
三重解密 (K1+K2+K3)
中间相遇攻击 (暴力破解)
系统运行日志 / 测试报告:
>> 基础解密: 885C -> 6A6A
>> 字符串扩展加密:
明文: Hello
密文: E6CE 5904 09C5
>> 双重加密: 明文 6A6A -> 密文 7BF3
>> 双重解密: 密文 7BF3 -> 明文 6A6A
>> 三重加密: 明文 6A6A -> 密文 B1E1
>> 三重解密: 密文 B1E1 -> 明文 6A6A

测试步骤：输入 48 bits 的三重密钥（K1、K2、K3 各 16 bits），采用 $C=E_{K3}(D_{K2}(E_{K1}(P)))$ 模式（EDE 模式）对 16bit 明文进行连续三次运算，解密时反向操作，观察结果。
原因：三重加密进一步将密钥空间扩展至 48 bits，能有效抵御中间相遇攻击。采用“加密-解密-加密”（EDE）模式，可以在 K1=K2=K3 时兼容单重 S-AES，同时打乱中间状态，确保算法具有极高的安全性。
结论：GUI 成功完成了 48bits 密钥的三重加解密操作，完美还原了原文。证明了三重加密算法及多轮密钥调度的逻辑严密、结果唯一正确。

4.5 第 5 关：工作模式



测试步骤：生成一段较长的明文和 16 bits 的初始向量（IV）。在 CBC 模式下，前一个密文分组与当前明文分组异或后再加密。加密完成后，人为篡改任意一个密文分组的数据，再点击解密，对比篡改前后的解密结果。
原因：CBC（密码分组链接）模式通过引入 IV 和反馈机制，使得相同的明文分组会生成不同的密文分组，消除了 ECB 模式下的明文图案特征。由于加密时的链接依赖，篡改某一密文块不仅会导致该块解密出错，还会导致下一个密文块解密完全乱码（错误传播特性）。
结论：CBC 模式成功加密并解密了长明文。篡改密文后，解密结果出现了对应块的比特错误以及后续分组的全面乱码。证明了 CBC 模式实现的正确性，并直观验证了其扩散机制与密文完整性保护的重要性。

五、 总结与反思

算法实现的严谨性：在实现 S-AES 时，状态矩阵的“按列排布”原则是易错点。我们通过比对教材标准向量，确保了底层算法的绝对正确。

安全性与分组长度的关系：通过中间相遇攻击实验，我们深刻认识到 16bit 分组存在极高的碰撞概率。这也证明了 S-AES 仅适用于教学，在现代密码学中是极其脆弱的。

模式与完整性的剥离：CBC 篡改实验让我们直观地看到了“加密”与“完整性保护”的边界。在生产环境中，必须结合 MAC/HMAC 才能防御篡改。

工程化思维：本次实验将算法内核与 GUI 界面完全解耦，实现了纯函数封装，不仅方便单元测试，也使得算法可以无缝接入其他后端系统。